# AgentProf: Semantic Profiling for AI Agents

**Anonymous submission**


## Abstract

AI agents increasingly orchestrate long-running, multi-step activities involving thousands to millions of interactions with users, tools, and system resources. To improve quality, safety, and cost efficiency of AI agent, developers need to determine where failures concentrate, which workflows trigger unsafe effects, and which task categories consume the most budget. To answer similar questions in traditional systems software, profiling attributes resource consumption to responsible entities, such as code paths, by aggregation. Yet existing agent observability tools are mainly for single-execution debugging or tracing instead of profiling, which makes it difficult to answer the above questions when the trajectories are becoming long and complex. Agent behavior differs from code execution, creating two challenges for profiling: the responsible entities are semantic categories such as task intent and workflow phase, not code paths, and agent trajectories lack the stable identifiers and structural hierarchies that make attribution possible. We propose a *semantic operation stack model* that adapts profiling to agent trajectories. Uniform *operations* represent all agent activities, and *operation stacks* replace the runtime call stack, enabling hierarchical attribution at different levels of granularity from the same data. We observe that an agent's task occupies a contiguous span of a trajectory and decomposes into contiguous subtasks, so task structure is a set of nested named intervals recoverable by finding only the transitions between them, and we design *recursive operation segmentation*: it splits a trajectory where the agent's responsibility changes, names the intervals on both sides, and recurses to supply the missing stack. AgentProf implements this in a profiler that compiles local agent trajectories into pprof-compatible profiles. On real trajectories and public datasets, our evaluation shows that semantic profiling effectively attributes cost and locates problems across agent trajectories.


## Introduction

AI agents (Yang et al. 2024; Anthropic 2025; Xie et al. 2024) orchestrate multi-step activities that span two layers, high-level intent (prompts, LLM calls, tool invocations) and low-level system effects (process spawns, file and network I/O). As agents evolve from short, scripted workflows into complex systems that run for hours and days, with a large number of interactions each, production teams accumulate many such trajectories over weeks or months.

To improve agent quality, safety, and cost efficiency, developers need to analyze operational behavior across trajectories and interactions. Which categories of tasks consume the most token budget? Where do failures concentrate across workflows? Which behavioral patterns trigger unsafe system effects? Answering these questions requires analysis and evaluation across many trajectories (Mazaheri and Mazaheri 2026; Lù et al. 2025), a task becoming costly for manual inspection or per-trajectory LLM evaluation (Zheng et al. 2023) at scale. In traditional systems software, profiling answers these questions by attributing resource consumption to responsible entities by aggregation, as distinct from single-execution debugging and tracing.

Yet existing agent tools support debugging and tracing but not profiling (Background). Some (LangChain 2024; Langfuse 2024; Arize AI 2024a; OpenTelemetry 2024) organize events into per-execution span trees, effective for diagnosing a single run but unable to aggregate by semantic category. Others (Datadog 2025; Laminar 2025) cluster inputs or extract structured signals, but characterize *input distributions* ("30% of users ask code questions") rather than *resource attribution* ("review tasks consume 40% of the token budget"), because tags at the request level do not propagate to downstream tool calls and system effects.

Adapting profiling to agent trajectories is challenging (Background). In traditional software, the responsible entities are code paths. Profiling is straightforward because function names are stable and runtime call nesting provides the attribution hierarchy (Background). Agent behavior differs fundamentally. To answer the questions above, the responsible entities need to be semantic categories such as task intent and tool operation, not code paths. Agent trajectories lack the two properties that make traditional profiling possible. Prompts are natural language, so two prompts expressing the same intent share no common string. Agent events have no runtime call-stack hierarchy because prompts, tool calls, and system effects are not nested by execution the way function calls produce stack frames: a sequence of prompts can be a task or multiple tasks, a task can be part of a bigger task.

Despite these differences, the fundamental profiling methodology transfers to agent trajectories: project resources onto responsible entities, assign stable tags, and attribute hierarchically. We propose a *semantic operation stack model* with two components. *Operations* are uniform records with string fields and additive measures that represent all agent activities (prompts, LLM calls, tool invocations, and system

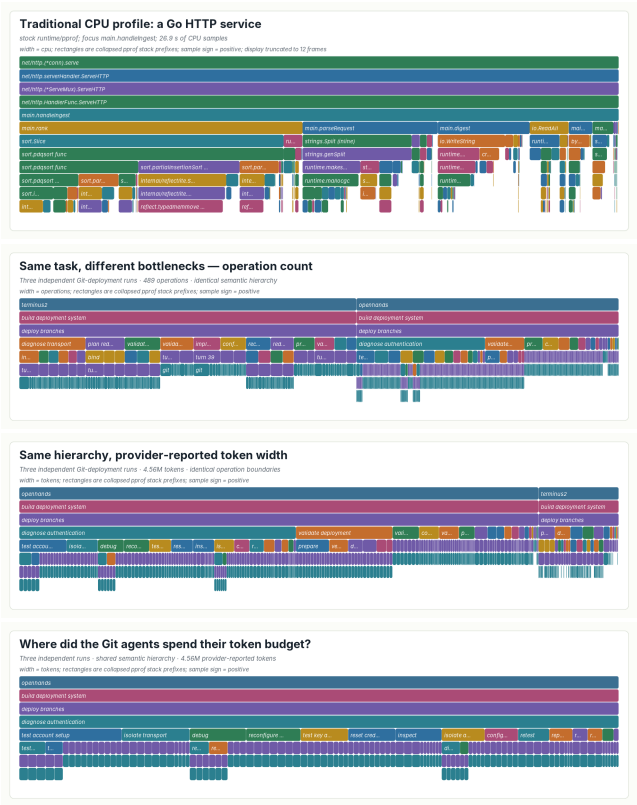


Figure 1: One renderer, four standard pprof profiles. *(a)* CPU time in a Go HTTP service under load, recorded by stock `runtime/pprof` and truncated to twelve frames for display. *(b, c)* All three independent agent executions of one Git-deployment task under one fixed operation hierarchy, produced by AgentProf and replayed under two additive measures: width is operation count in (b) and provider-reported tokens in (c). The hierarchy, its boundaries, and its names are identical in both; only the measure changes. *(d)* The same token profile focused on the shared `diagnose authentication` responsibility, which recursion expands into account, transport, credential, and retest operations. Every panel is read the same way—width is aggregate cost, a parent's cost contains its children's, and identical stacks fold—and only the meaning of the frames differs between (a) and (b–d).

effects) without type-specific objects. *Operation stacks* provide hierarchical attribution, replacing the runtime call stack. Choosing different fields changes the attribution granularity: the same operations can be attributed by task category, by execution phase, by session, or by action type, and one fixed hierarchy replays under any additive measure (Figure 1).

We implement this model in AgentProf, an offline profiler that compiles local agent trajectories into pprof-compatible profiles (Figure 2). One algorithm supplies the tags the trajectory does not carry, and its design follows from one observation about agent work: a task occupies a contiguous span of a trajectory and decomposes into contiguous subtasks, so task structure is exactly a set of nested named intervals, and recovering it requires only finding the transitions between them. This makes the problem segmentation rather than classification: no predefined taxonomy is required, and each branch's depth emerges from the content instead of a fixed schema. *Recursive operation segmentation* therefore reads a trajectory, cuts it at the points where the agent's responsibility changes, names the intervals on both sides, and recurses, so short action-first names give agents the stable identifiers that traditional profiling gets for free from function names. The segmentation contract is backend-neutral: an agent, a plain language model, or a statistical rule can produce the same marks. Across all 405 released CodeTraceBench trajectories, recursive segmentation agrees with human stage annotations at 0.764 ordinary $B^3$ F1, versus 0.663 for a statistical baseline and 0.541 for raw actions. On three complete localization benchmarks, the profile raises MAP over each benchmark's own diagnostic by .031, .107, and .117, and as a reading index it guides a strong reader to equal ranking quality while opening 53.0% instead of 65.0% of the source evidence. One fixed hierarchy replays under operation-count and token measures with exact conservation, exposing bottlenecks that a count view understates by more than a factor of two.

This paper makes three contributions:

1. **Semantic operation stack model.** We identify the missing profiling layer in agent observability and propose a model of uniform operations and query-time operation stacks (Background and Design).
2. **System: AgentProf.** A profiler that implements this model through a backend-neutral recursive-segmentation contract, producing pprof-compatible output (Implementation).
3. **Evaluation.** We evaluate profiler output against independent ground-truth annotations that stay hidden from the profiler, on eight public benchmarks and three real trajectory populations (Evaluation).

## Background and Motivation

### LLMs and AI Agents

A typical agent trajectory interleaves two layers of activity. At the intent layer, the agent receives a user prompt, issues LLM calls, and invokes tools (code execution, web search, file editing). Each tool invocation triggers system effects at the lower layer: process spawns, file reads and writes, and network requests. A single trajectory may contain hundreds of such intent-effect cycles, and a production team accumulates many trajectories over time.

### System Profiling

Profiling attributes resource consumption to responsible entities by aggregation, as distinct from single-execution debugging and tracing. In traditional software, the profiling pipeline works as follows: a profiler periodically samples execution state, attaches each sample to the current call stack, and merges samples with identical stacks. Flame graphs (Gregg 2011) and pprof (Google 2024b) call graphs visualize the

result, where width or node size represents aggregate cost. Perfetto (Google 2024a) generalizes this with SQL-based analysis and derived events. These tools support flexible aggregation, but they all assume stable function names and a runtime call stack. Pprof's `tagroot`/`tagleaf` options can add label-derived pseudo-frames, but only on top of an existing execution stack that agent trajectories do not have.

## A Motivating Case

Consider three independent agent attempts at one real Git-deployment task, none of which delivered the requested password-authenticated endpoint. Figure 1 places a conventional CPU profile beside the semantic profile of those three runs. Both are standard pprof profiles drawn by one renderer and read by identical rules: width is aggregate cost, a parent contains its children, and identical stacks recorded at different moments fold into one frame. What differs is where the frames come from. A call stack supplies `net/http.(*conn).serve` at every sample for free; nothing in an agent trajectory supplies `diagnose authentication`, because the three runs express that intent through different prompts and different shell commands. Once that name is constructed, the profile answers the engineer's question directly.

The segmentation backend builds one hierarchy over all three executions (489 operations, 4,558,192 provider-reported tokens, semantic depth five with no fixed limit; Figure 1). The repeated `diagnose authentication` subtree is 21.47% of the task by operation count but 46.15% by tokens, and drilldown shows all three executions verifying substitute transport paths without ever establishing the endpoint—nearly half the budget went to one never-successful diagnosis that a count view understates by more than half. A same-input control replays identical operations and weights under native, coarse-action, and semantic organizations: the responsibility scatters across 105 native calls and six coarse action kinds (largest branch 39.42% of its tokens); only the semantic organization reunites it into one focusable cross-run path. This paper's evaluation returns to this case under RQ1 and asks whether the same behavior holds at population scale.

## Challenges for Agent Profiling

Existing tools (LangChain 2024; Langfuse 2024; Arize AI 2024a; OpenTelemetry 2024) record agent events as per-execution span trees, effective for debugging a single run. AgentSight (Zheng et al. 2025) bridges intent and system-effect layers by correlating eBPF-captured system events with intercepted LLM traffic, but the resulting recordings still lack aggregate profiling.

Adapting profiling to agent trajectories faces two core challenges. The first is that the responsible entities differ. In traditional software, profiling attributes cost to code paths, but in agent trajectories the relevant entities are semantic categories such as task intent and workflow phase. Existing tools (OpenTelemetry 2024; Arize AI 2024b) do not capture these categories because prompts are natural language: two prompts and tool invocations expressing the same intent may be different, so the profiler cannot aggregate them the way it aggregates identical function names. The second challenge is that agent trajectories have no runtime hierarchy for attribution. In CPU profiling, the call stack determines attribution at every level: `malloc`'s cost belongs to `parse`, to `process`, and to `main` simultaneously. A sequence of prompts may constitute one task or several, and a task may be part of a larger workflow, but no runtime mechanism marks these boundaries or determines at which granularity to attribute cost. We quantify this in RQ1 (Evaluation).

## Design Requirements for Agent Profiling

Traditional profiling relies on three mechanisms that the call stack provides automatically. Agent profiling needs all three but must achieve them without a call stack:

**R1: Cross-layer resource projection.** In traditional profiling, each sample is taken inside a call context, so resource consumption is automatically attributed to the code path that caused it. Agent activities span two layers: intent (prompts, LLM calls) and system effects (file I/O, process spawns). The profiler must connect system-layer resource consumption to intent-layer semantic categories.

**R2: Stable tags for folding.** In traditional profiling, function names are stable identifiers that enable folding identical stacks. Agent prompts are natural language, so the profiler must derive short, stable, repeatable tags before folding.

**R3: Hierarchical attribution.** In traditional profiling, the runtime call stack provides the attribution hierarchy: function calls nest, and folding identical stacks produces the tree that profiles visualize. Agent events are not nested by execution, so the profiler must construct attribution hierarchies from the data.

# Design

The three requirements (cross-layer resource projection, stable tags, and hierarchical attribution) drive the design. Operations (Design) satisfy R1 by representing intent and system-effect activities in a single record with tag inheritance, so intent-layer tags propagate to the system effects they trigger. *Recursive operation segmentation* (Design) satisfies R2 and R3 together: it derives short, stable interval names from trajectory content and nests those intervals into the attribution hierarchy that replaces the runtime call stack. *Operation stacks* (Design) then attribute resources at every level of that hierarchy at query time, so the same data supports multiple views without rebuilding the input. The profiling pipeline has four stages: (1) parse raw trajectories into operations, (2) segment trajectories into named nested intervals (or apply mapping rules), (3) project each operation onto a stack frame sequence chosen at query time, and (4) fold operations with identical stacks, summing their weights.

## Semantic Operation Stack Model

Because agent activities span both the intent and system-effect layers described in Background, a profiler should not distinguish between them in the data representation. AgentProf therefore represents every activity as a single abstraction, the `operation`, a weighted record with

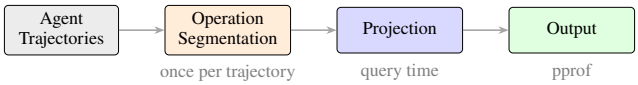


Figure 2: Data-flow and algorithm pipeline. Local agent histories enter through `agent-session` parsing; public datasets enter as operation JSONL. Both paths produce uniform operations. Recursive operation segmentation, projection, and folding are algorithms (segmentation runs once per trajectory; projection and folding run at query time) applied identically to both sources.

string fields and additive measures. Each operation carries string fields (`project`, `agent`, `session`, `prompt_tag`, `kind`, `model`, `path`, `domain`, `status`, ...) and additive measures (token count, duration, event count). A prompt, an LLM call, a tool invocation, a file read, a GUI click, and a process event are all operations with the same schema, distinguished only by which fields carry values.

An `operation stack` provides hierarchical attribution for agent trajectories, replacing the runtime call stack. In CPU profiling, the call stack determines which code paths share responsibility for a resource cost. An operation stack serves the same role: an ordered list of fields $[f_1, f_2, \ldots, f_k]$ projects each operation $o$ to a frame sequence $\langle o.f_1, o.f_2, \ldots, o.f_k \rangle$, and operations with identical sequences are merged, summing their weights. Unlike a call stack, the fields are chosen at query time, not determined by execution. The same operations can therefore be attributed as a flat task summary, a per-session tree, a semantic phase/action hierarchy, or a trajectory using the dataset's own annotations, and changing the fields changes the attribution without changing the data. Sessions and spans are optional operation fields, not fixed hierarchy levels, so the same data supports both debugging views (include `session`) and aggregate profiling views (exclude it). The frame sequence need not come from fields alone: recursive segmentation (Design) supplies each operation's covering interval-name chain as a variable-depth frame sequence, and field lists and interval paths compose in one stack. A view is a triple $(\varphi, \sigma, w)$: a predicate $\varphi$ selects which operations participate, a stack function $\sigma = [f_1, \ldots, f_k]$ maps each operation to its frame sequence, and a weight function $w$ assigns a nonnegative measure. Four built-in views cover common questions: `tokens` (LLM calls weighted by token count), `time` (all timed operations weighted by duration), `files` (file operations weighted by event count), and `network` (network operations weighted by event count).

## Recursive Operation Segmentation

The algorithm rests on one structural intuition: an agent's task occupies a contiguous span of the trajectory, and it decomposes into smaller contiguous subtasks. Task structure is therefore exactly a set of nested named intervals, and recovering it only requires finding the transition points between them.

The core algorithm is segmentation, not labeling: instead of assigning each step a class from a predefined taxonomy, the backend reads a trajectory and finds the transition points where the agent's current responsibility changes. At each transition it cuts the trajectory and names the intervals on both sides; it then recurses into each interval, cutting again wherever a finer responsibility change remains, and stops when an interval reads as one coherent piece of work. Formally, a trajectory is an ordered turn sequence $T = (t_1, \ldots, t_n)$, and a segmentation is a set $S$ of named intervals over $T$ that is nested (any two intervals are disjoint or one contains the other), covers every turn, and contains the whole-session interval as its root. The backend computes $S$ by one recursive rule: $\textsc{Segment}(I)$ selects zero or more transition points inside interval $I$, which partition $I$ into consecutive child intervals; each child receives a short action-first name and is segmented recursively; selecting no transition terminates the branch and declares $I$ one operation. A turn's operation path is the name chain of the intervals containing it, from the session root to its innermost interval; equal paths fold across sessions. Segmentation is the right primitive for three reasons. First, transitions are what a trajectory actually exhibits: agent work has no predefined taxonomy, but a shift in what the agent is trying to do is visible in the source content itself. Second, cuts compose: nested intervals form a hierarchy whose depth emerges per branch from the content rather than from a prescribed schema. Third, naming intervals rather than steps yields identifiers at the granularity where recurrence lives, so the same short action-first name (one to three words) reappears across sessions and folds. Concretely, in one Git-deployment execution the backend cuts the session into `build deployment system` and, inside it, cuts again where the agent stops writing configuration and starts debugging access, yielding `deploy branches` and `diagnose authentication`; a turn inside the latter carries the path `build deployment system > diagnose authentication`, and because the same two names reappear in the other two executions, their costs fold together in Figure 1. Any backend able to find transitions and name intervals—an agent, a plain language model, or a statistical rule—implements the same contract; the evaluated backend is a frontier code agent invoked once per trajectory with one fixed instruction. That instruction is the complete backend interface:

> *Input:* one trajectory's source-only packet—each turn's visible prompt, command, and output summary, with no stage labels, outcomes, or scores.
> *Action:* read the packet once; emit a mark wherever the responsibility changes.
> *Output:* sparse complete-path marks, for example
> ```
> turn 1: [deploy git server]
> turn 4: [deploy git server, diagnose
> SSH access]
> turn 9: [deploy git server, diagnose
> SSH access,
>   test PAM]
> turn 15: [deploy git server, verify web
> endpoint]
> ```
> —one line only where the path changes, free depth per branch, names of one to three action-first words.

Turns between two marks inherit the preceding path, so the marks are a complete segmentation despite being sparse.

One ordinary format retry is allowed; afterwards the deterministic prefix repair and canonicalization replay leave the downstream pipeline unchanged.

## Implementation

AgentProf is an offline, post-hoc profiler implemented as a Rust CLI (∼9.8K LOC, Figure 2). It reads Codex and Claude Code local JSONL history files and AgentSight (Zheng et al. 2025) recordings for system effects, and reconstructs prompts, LLM calls, tool invocations, and their triggered file and network effects. The backend works in a three-file annotation workspace (`trace.jsonl`, `annotation.json`, `stacks.folded`); the CLI validates that marks are nested, noncrossing, and cover every source node, then emits one standard pprof protobuf profile that existing tools such as `go tool pprof` read directly. Projection is deterministic: each source node contributes the agent, its covering semantic operation path, and its retained LLM-call and tool-call evidence, with raw session and prompt identities kept as pprof labels rather than visible frames, so equal semantic prefixes fold across sessions; switching the additive measure replays the same annotation and changes only stack widths.

Source-specific adapters preserve replay-stable node IDs, source content or readable references, native parent links, input order, and additive measures, and materialize structured operation JSONL as adapter-defined atomic source nodes without inventing missing prompt or call relations. The backend reads `trace.jsonl` and writes only `annotation.json`, which maps source start IDs to `tag`, semantic `parent`, and exclusive `next`; after validation AgentProf atomically updates derived paths and the folded aggregate. Nonblocking hierarchy warnings flag a recursive operation with only one explicit semantic child, a large flat fan-out with little recursive refinement, or an optional semantic leaf spanning at least eight tool calls—exposing redundant, prematurely flat, or coarse annotations without forcing artificial depth or blocking generation.

**Non-LLM recurrence backend.** The inducer scores adjacent action transitions by normalized pointwise mutual information (Bouma 2009) and calibrates a label-free cutoff with occurrence-weighted two-means (MacQueen 1967), treating detailed recurrence as continuity that can remove but never add a coarse boundary; an optional reference-calibrated mode fits one scalar cutoff on disjoint reference groups.

**Privacy.** Workspace construction, validation, folding, materialization, and replay run entirely on the local machine, and profiles carry only short semantic names, bounded text previews, and numeric measures as labels. Automatic annotation with a hosted model backend sends the preview-truncated packets to that provider; annotation with the local recurrence backend or a local model keeps every byte on the machine, and nothing else leaves the machine unless the user shares the profile.

## Evaluation

The evaluation addresses four research questions. **RQ1:** Does semantic profiling improve resource attribution? **RQ2:** Does profiler output correspond to real problems? **RQ3:** How accurate are the tags? **RQ4:** What is the profiling cost? Together they form one chain: semantic responsibility can be recovered accurately (RQ3), the recovered hierarchy aggregates across runs while conserving every additive measure (RQ1), the aggregate corresponds to real problems and reduces the evidence a reader must open (RQ2), and construction and replay costs are measured and practical (RQ4).

We evaluate three data classes. *Real inputs*: 41 long-horizon coding-agent sessions (three of which repeat one Git-deployment task and form the RQ1 case), 440 mixed-outcome web-agent trajectories (Lù et al. 2025), and 42 development sessions from the authors' workstation. *Annotated trajectories*: 405 CodeTraceBench trajectories with 20,866 operations and 2,948 human-verified stages (Li et al. 2026), 287 OSWorld-Human sessions (WukLab 2025), 1,012 AgentBoard goals (Ma et al. 2024), and 2,737 published action labels (Bouzenia and Pradel 2025). *Problem-localization benchmarks*: the complete AgentProcessBench, HINTBench, and TraceElephant workloads, where every trajectory carries an independently annotated faulty step, over 27,346 operations (Fan et al. 2026; Wang et al. 2026b; Chen et al. 2026). All annotations, outcomes, and scores stay hidden from every backend until its output is frozen. Mean operations per trajectory are 8.5 (AgentProcessBench), 13.9 (OSWorld-Human), 24.0 (HINTBench), 27.1 (TraceElephant), and 51.5 (CodeTraceBench), so the benchmark trajectories are short-to-medium horizon while the 42-session workstation population—whose longest sessions span tens of hours—supplies the long-horizon regime; the RQ4 union covers 27,765 operations.

### RQ1: Does Semantic Profiling Improve Resource Attribution?

*Setup.* We test one necessary consequence of the claim: whether a fixed semantic hierarchy reveals materially different bottlenecks when the additive resource measure changes. The source population contains 41 real long-horizon agent trajectories, 3,146 source-native turns, and 5,750 operations, including three independent executions of one Git-deployment task whose 735 source nodes receive 96 recursive annotations; the resulting hierarchy reaches semantic depth five without a fixed limit, and the CLI reports no unary or flat-fan-out warning and 27 advisory coarse-leaf warnings.

**Use case 1: why did three runs of one task all fail?** This is the case previewed in Section . A stock-pprof focus query retains all three executions rather than selecting one trace, and Figure 1 replays that one fixed hierarchy under both additive measures. The focused task contains 489 operations and 4,558,192 provider-reported tokens from OpenHands/Claude, OpenHands/DeepSeek, and Terminus2/DeepSeek: by count Terminus2 contributes 56.24% and OpenHands 43.76%, while by tokens OpenHands contributes 86.62%. The repeated `diagnose authentication` operation directly contains 97 operations and 1,936,828 tokens, and its complete recursive subtree contains 105 operations and 2,103,587 tokens (21.47% and 46.15% of the focused task). The same unchanged hi-

erarchy also maps all 489 evidence rows under a defined elapsed-time width and exactly conserves 3,982 seconds, of which the subtree holds 1,492 seconds (37.47%). Its attributed share therefore rises from 21.47% by count through 37.47% by time to 46.15% by tokens: changing only the measure moves the share by more than a factor of two, and the count view attributes the task mainly to Terminus2 while the token view attributes nearly half of it to the authentication responsibility, which source drilldown resolves to specific unfinished work.

As a post-hoc organization control, we replay the same 489 source operations and both weights over common `project, agent` prefixes and `call, tool` leaves with three middle organizations—native (`session, prompt`), coarse (`action kind, raw action`), or semantic—so all six profiles conserve exact mass and open in stock pprof. The SSH members scatter across 105 native calls and span six coarse action kinds (largest branch 39.42% of their tokens), with 102 of 105 merely `run` alongside 97 unrelated operations; only the semantic organization yields one focusable cross-run responsibility that an operation-count view understates. Separately, real AgentSight (Zheng et al. 2025) eBPF recordings replay the same way: the R114 population folds all 1,520 process and file effects under task responsibilities through the recorded wrapper tool with exact conservation, demonstrating the system-effect layer end to end.

**Use case 2: where did this project's agent budget go?** A developer profiles the 42 sessions their own coding agents produced while building AgentProf and this paper, asking what weeks of agent work actually spent tokens doing. The population is 18 Codex and 24 Claude Code sessions recorded on the authors' workstation, including ten long-horizon sessions whose longest span tens of hours and hundreds of prompts (Figure 3). In this native no-sudo local-history scenario, one `-workspace-out` invocation initializes the annotation workspace directly from the raw session logs, and the adapter materializes 10,423 source nodes: 42 sessions, 1,252 prompts, 5,620 LLM calls, and 3,509 tool calls carrying 1,380,863,014 bounded provider token components. The fixed automatic instruction, identical to the AgentRewardBench run, makes one pass, produces 1,737 semantic annotations at depths two through four, covers all 1,294 mandatory session and prompt scopes, and incurs zero backend failures across 42 batches; the operation-count and token profiles conserve exactly 3,509 and 1,380,863,014.

The same self-profile hierarchy replays under FILE-READ, FILE-WRITE, and NETWORK-target widths with exact conservation of 737, 31, and 61 target references (Figure 4); in the FILE-WRITE view, source-linked LLM and tool frames remain beneath each semantic operation, and each created file appears as an exact drilldown leaf beneath the operation that created it.

The answer is a broad profile rather than a single sink: the largest token path, `refine paper > align evaluation`, holds only 1.735% of mass. Token mass stays mostly at the mandatory prompt depth (70.4%), reflecting genuinely many-tasked development sessions under the fixed one-pass protocol, whereas operation mass resolves more deeply (43.9% at depths three and four) exactly where repeated engineering work concentrates. The three longest sessions keep distinct dominant responsibilities—evaluation alignment, evidence inspection, and merge/conflict resolution—so long-session structure is preserved rather than averaged away. With three workers the annotation critical path is 44.6 minutes and consumes 15,231,328 reported input tokens (13,112,320 cached) plus 311,097 output tokens, and validation completes in 0.211 seconds; this case establishes descriptive feasibility on real long-horizon sessions without outcome labels.

Beyond single corpora, the two additive measures agree strongly at population scale: replaying the frozen 440-session hierarchy (7,229 operations and 51,904,621 tokens, both conserved exactly) ranks every operation once by count and once by tokens, with mean per-task Kendall's tau-b (Kendall 1938) 0.886 (interval [0.857, 0.915]; Spearman's rho (Spearman 1904) 0.935) and 10 of 77 rankable tasks below 0.7. *Takeaway.* One reusable hierarchy attributes multiple resources with exact conservation; as the motivating case's same-input control shows, it is the only tested organization that reunites a cross-run responsibility, and the minority of tasks where count and token rankings diverge is precisely where choosing the measure changes the engineering decision.

## RQ2: Does Profiler Output Correspond to Real Problems?

*Setup.* Each benchmark trajectory has one annotated faulty step; a method outputs a ranking of that trajectory's operations, scored by average precision (AP approximates the reciprocal rank of the faulty step) and averaged as MAP (Robertson 2008). We run the complete AgentProcessBench, HINTBench (the complete released test snapshot, 536 of the paper-reported 629 trajectories), and TraceElephant workloads; operations, target-blind paths, benchmark predictions, and scoring are fixed before test targets are loaded. Each query is one trajectory containing an annotated target, and we report non-interpolated per-query AP averaged over 614, 400, and 220 target-bearing queries; all 522 zero-positive trajectories are consumed for population coverage but excluded from MAP because AP is undefined without a relevant item. The *direct diagnostic* is the per-operation output of a benchmark-native process judge (AgentProcessBench risk units) or trajectory localizer (HINTBench, TraceElephant), which for TraceElephant reads the full trace and reference answer before naming the responsible agent and decisive step. *Direct-only* ranks by that diagnostic alone; *Direct+AgentProf* ranks lexicographically by (direct diagnostic, target-blind group score) and can therefore only refine exact diagnostic-score ties; *Direct+Raw+Evidence* is the information-matched control that groups by raw action instead of semantic names, with identical source-kind, tool/call, and outcome suffix and aggregation; *AgentProf-only* drops the benchmark diagnostic (Table 1).

*Results.* Direct+AgentProf beats Direct-only by .031 [.024, .039], .107 [.093, .120], and .117 [.088, .148], using 10,000 paired resamples of trajectory clusters within benchmark-defined strata, so every paired interval lies above zero (Fig-

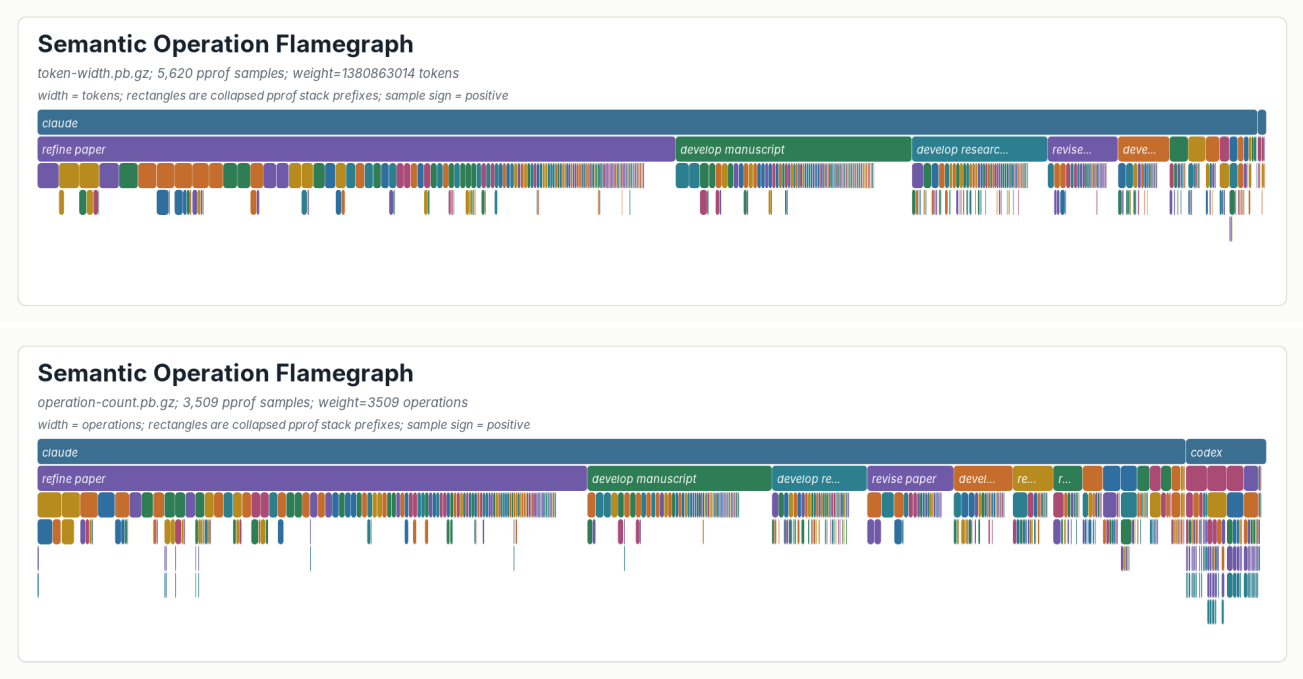


Figure 3: The complete 42-session population under one fixed semantic hierarchy. Width is provider-reported tokens (top) or operation count (bottom); second-level frames are the annotated responsibilities (`refine paper`, `develop manuscript`, `develop research`, ...), with deeper refinements and source labels available for drilldown in stock pprof.

| Workload | Direct+ AgentProf | Direct+Raw +Evidence | Direct only | AgentProf only |
|---|---|---|---|---|
| AgentProcessBench | **.894** | .893 | .863 | .791 |
| HINTBench | .517 | **.518** | .411 | .432 |
| TraceElephant | **.326** | .324 | .209 | .259 |

Table 1: MAP over the complete RQ2 populations (614, 400, and 220 target-bearing queries). Higher is better.

ure 5). The information-matched control reaches .893, .518, and .324, with candidate-minus-baseline intervals [-.0003, .0029], [-.0116, .0103], and [-.0247, .0280]; this experiment therefore does not establish that the semantic prefix ranks targets better when both views retain the same source evidence. The tie follows from the mechanism: the per-step anomaly signal lives in the source evidence that both views share by construction, so grouping plus evidence—not the names—drives this ranking gain. The semantic prefix's distinct, separately measured roles are cross-run attribution (RQ1) and directing a reader's attention, which the following study measures.

**Profile-guided reading on TraceElephant.** On the complete 220 target-bearing queries, a fixed external Grok-family CLI reader receives target-blind packets—task text, operation IDs, and source-visible content—with unranked operations appended in original order deterministically and one single-turn call per stage. Full-trace reading reaches MAP .502 at a mean 12,615 input tokens per query, versus .209 for the benchmark's Direct-only diagnostic and .326 for Direct+AgentProf. A two-stage profile-guided variant first shows only the semantic operation skeleton, with no source content, and the reader selects at most five groups; stage two then opens only those groups' evidence. It reaches MAP .455 while opening 53.0% of the source content, and its stage-one selections never fell back to a default: the five-group budget saturated on 99.5% of queries, re-parsing the uncapped stage-one responses shows the reader proposed more than five groups on zero of 220 queries, and every missed target group was entirely absent from its ordered selection rather than cut off by the budget. Under an information-matched raw-action skeleton, ranking quality is statistically unchanged (MAP .465; paired delta $+.010$ $[-.021, +.042]$), but the reader opens significantly more content: 65.0% versus 53.0%, paired delta $+.120$ $[+.103, +.137]$, and 2.80 [1.96, 3.60] more evidence operations. Semantic naming's measured contribution in this regime is therefore attention concentration at equal quality. A per-query full read is also bounded by the model context window—the 4,558,192-token Git population cannot be read whole—whereas skeleton-guided reading stays available at any trace length once the profile exists, and the reader is query-specific while the hierarchy is constructed once and replayed across queries and measures.

**Use case 3: what do failing runs do differently?** A team has 440 real web-agent runs over 125 tasks, where every task contains both successful and failed attempts, and asks what behavior separates the two—without reading 440 transcripts.

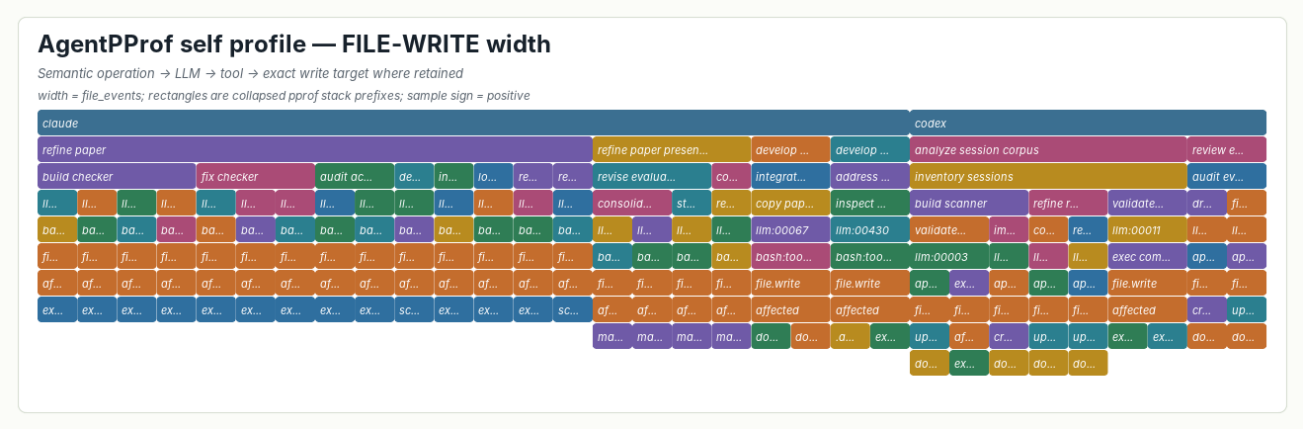


Figure 4: FILE-WRITE view of the same self-profile hierarchy. Width is target-reference count, with each created file preserved along the semantic operation → LLM → tool → exact write-target path.

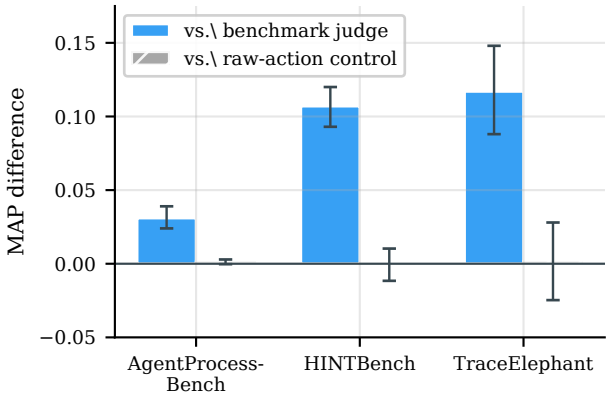


Figure 5: Paired MAP differences over the complete RQ2 populations, with 95% intervals from 10,000 trajectory-cluster resamples of unrounded per-query APs. Adding the target-blind profile to each benchmark's own judge improves ranking on all three workloads (left bars, intervals entirely above zero), whereas the semantic prefix and the information-matched raw-action control are indistinguishable (right bars, intervals spanning zero). Grouping plus retained evidence, not the semantic names, drives this gain.

We aggregate the complete mixed-outcome population (Lù et al. 2025) rather than one favorable trace pair: the 440 trajectories yield 338 bad–good pairs (24/102/144/68 from AssistantBench, VisualWebArena, WebArena, and WorkArena), with the 202 successful and 238 unsuccessful sessions reusing across pairs under pair-occurrence weighting. Before opening outcomes or expert annotations, three automatic Agent workers produce 2,131 sparse recursive annotations over all 7,229 source operations, and the resulting hierarchy reaches semantic depth four before its LLM-call and tool-call evidence leaves. Formally, the signed profile is the difference of two nonnegative profiles, bad minus good; each side satisfies the model's nonnegative additive-measure definition, and pprof renders the subtraction as positive and negative sample values. Folding all 338 pairs into that one signed profile answers at a glance (Figure 6): the aggregate contains 7,366 bad-side and 3,780 good-side operation occurrences, recovery work occupies 44.6% of failed-side versus 12.0% of successful-side occurrences (completion 1.8% versus 5.1%), and the recursive recovery focus separates verification challenges, repeated searches, mistaken navigation, product or record retries, and repository inspection beneath the same responsibility, with source labels identifying the contributing session and call. We then test whether this visible structure corresponds to an independent problem rather than merely reflecting more steps: per-trajectory recovery exposure—the fraction of source operations whose path contains the shared recovery responsibility, computed per unique trajectory after deduplicating pair reuse and therefore unaffected by pair multiplicity—predicts consensus expert looping labels on 435 trajectories at AP .634 versus prevalence .398 (AP-minus-prevalence interval [.181, .293] over 10,000 task-cluster draws). A registered fixed-chain repeated/error projection obtains AP .656, and the recursive-minus-fixed interval [-.107, .061] does not establish detector superiority; the two projections answer different questions, since the fixed indicator only flags where repeated or error steps occur, whereas the recursive profile decomposes the same responsibility into verification, search, navigation, retry, and inspection with session and call drilldown. Failed runs spend nearly half their steps re-trying and re-navigating, and the structure the profile surfaces matches what human experts independently marked. *Takeaway.* Profiles add localization signal on top of existing judges; the semantic names buy evidence efficiency rather than ranking accuracy; and failure structure surfaced by the profile matches independent human labels at collection scale.

## RQ3: How Accurate Are the Tags?

*Setup.* Gold structure is the 2,948 human-verified stages over all 405 CodeTraceBench trajectories. $B^3$ F1 (Bagga and Baldwin 1998) asks whether operations that belong to-

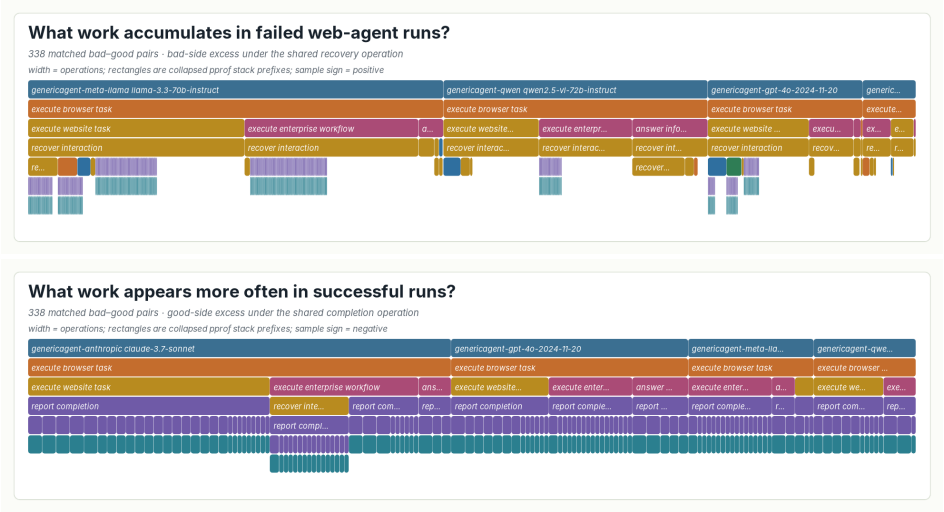


Figure 6: One signed profile over all 338 bad–good pairs, read from both sides. The signed profile is the difference of two nonnegative profiles, bad minus good, which stock pprof renders as positive and negative sample values. *Top:* positive samples under `recover interaction`, 3,286 bad-side versus 455 good-side occurrences. *Bottom:* negative samples under `report completion`, 135 bad-side versus 191 good-side occurrences. These are aggregate diagnostic differences, not causal effects.

gether end up grouped together; exact adjacent-boundary F1 (Ruokolainen et al. 2016) asks whether cuts land exactly where humans cut. Baselines receive identical inputs (Table 2). Each backend output is evaluated at the level it predicts: literal names by accuracy and macro-F1 (Lewis et al. 2004), permutation-invariant partitions by V-measure (Rosenberg and Hirschberg 2007) or ordinary $B^3$, and adjacent interval boundaries by exact precision, recall, and F1; for legacy field-valued datasets a mechanical adapter converts each predicted group into the same interval-annotation contract before AgentProf folds the original additive weights. Direct Agents read complete source-only packets once and emit sparse complete-path marks over 17,148 source-native turns without access to official stages or outcomes before materialization: 4,496 marks at semantic depth one (3), two (2,873), three (1,588), or four (32), with no prompt requesting a depth. The adopted marks' name replay maps 3,895 open-vocabulary operation IDs to 783 stable action-first canonical IDs, re-expanding all operations from the unchanged marks, preserving the temporal occurrence and adjacent-boundary vectors exactly, and leaving no adjacent display-path collision; nonadjacent and cross-session equal names intentionally merge so recurring work folds in pprof.

*Results.* Direct segmentation reaches 0.764 $B^3$ F1 and 0.480 boundary F1, beating the strongest automatic baseline by +0.101 [0.087, 0.116] and +0.214 [0.193, 0.235] (paired task-cluster intervals); the marks conserve exactly 20,866 operations and 494,862,929 tokens. The residual boundary error is over-segmentation rather than missed transitions (recall 0.626, precision 0.389): predicted groups remain largely pure subsets of gold stages ($B^3$ precision 0.793), so the dominant failure subdivides work rather than merging unrelated responsibilities. The contract generalizes across backend families: on 287 OSWorld-Human sessions a supervised boundary predictor (McCallum and Nigam 1998) reaches 0.739 boundary F1 / 0.816 $B^3$ F1 and the no-LLM recurrence inducer 0.680 / 0.786 (strongest control: 0.645 / 0.678), and a target-blind TF-IDF/K-Means backend reaches V-measure 0.557 on nine Mind2Web sessions and 0.815 on 100 ScienceWorld sessions (Deng et al. 2023; Wang et al. 2022). For closed-label literal tags, a fixed evaluation-only Qwen3.6-27B (Qwen Team 2026) backend classifies all 1,012 AgentBoard goals (Ma et al. 2024) into nine task families at 0.695 macro-F1 and 0.733 accuracy (majority baseline 0.044 / 0.248), and a backend-level adapter classifies all 2,737 published action labels from 120 AutoCodeRover, OpenHands, and RepairAgent trajectories (Sajadi et al. 2026; Bouzenia and Pradel 2025) at 0.498 macro-F1 and 0.628 accuracy (majority baseline 0.061 / 0.323). *Takeaway.* Recursive segmentation is the most accurate tested automatic constructor on both structural axes, and non-LLM backends implementing the same contract remain viable when no model is available.

| Method | $B^3$ P | $B^3$ R | $B^3$ F1 | Bound. F1 |
|---|---|---|---|---|
| Direct Agent segmentation | 0.793 | 0.736 | **0.764** | **0.480** |
| Multi-resolution recurrence | 0.782 | 0.575 | 0.663 | 0.266 |
| Native source tree | 0.975 | 0.249 | 0.397 | 0.259 |
| Source-native turn | 0.983 | 0.221 | 0.361 | 0.246 |
| Raw-action grouping | 0.891 | 0.388 | 0.541 | — |

Table 2: Agreement with human stages on all 405 CodeTraceBench trajectories.

## RQ4: What Is the Profiling Cost?

*Setup.* Cost separates into a one-time automatic segmentation pass per corpus and deterministic construction/replay afterwards. The profiling core—parsing operation JSONL,

constructing stacks, folding weights, and serializing the profile—is measured by comparing the fixed semantic hierarchy against the raw hierarchy on identical inputs over three runs across four complete public workloads and their union, using `agentpprof 0.2.37` on a 24-core Intel Core Ultra 9 285K with 125 GiB RAM and Linux 6.15.11. This microbenchmark exercises the generic parse/fold/serialize core on public field-stack workloads; the annotation-mark representation's deterministic pipeline is timed separately. *Results.* Segmenting all 405 CodeTraceBench trajectories consumes 12,050,384 input and 231,886 output tokens in 2,215.9 s of active backend time; constructing the 405 source-only packets the direct backend reads takes 501.64 s median, and after annotation the full deterministic downstream pipeline takes 11.516 s. A fully instrumented end-to-end pass on the complete 440-session population completes all 12 outcome-blind batches in 3,521.6 s on a fixed two-worker schedule (58.7 minutes; summed worker time 6,661.7 s), consuming 12,039,417 actual input tokens (10,929,408 reported cached, 90.8%) and 312,433 output tokens—27,362 input and 710 output tokens per session—while one fresh complete pass over the three-session Git population takes 466.9 s and 832,544 actual input tokens. With marks fixed, constructing a 27,765-operation union profile takes 1.16 s at 465.2 MiB peak memory (0.0418 ms per operation, $R^2 = 0.9997$; 23,935 operations/s), 190 ms (19.6%) more time and 5.25 MiB (1.14%) more peak RSS than raw-action grouping, and switching the additive measure is a replay at the same cost. Deterministic materialization of the full 440-session population takes 0.26 s for operations and 0.25 s for tokens, so construction cost is dominated by the automatic backend while materialization stays sub-second. *Takeaway.* A corpus pays minutes of model time once; every later view, measure switch, and drilldown costs about a second.

## Related Work

**Agent observability.** Agent observability tools (LangChain 2024; Langfuse 2024; Arize AI 2024a; OpenTelemetry 2024; Arize AI 2024b; Dong, Lu, and Zhu 2024; Balusu 2026; Wang et al. 2026d) aggregate spans, metadata, and evaluations, and focus on single-execution debugging. Datadog Patterns and LangSmith Insights derive cross-trace hierarchies and roll up metrics, NeMo profiles instrumented workflows, and OpenTelemetry Profiles links profiles to traces (Datadog 2026; LangChain 2026; NVIDIA 2026; OpenTelemetry 2026). Datadog LLM Observability (Datadog 2025) and Laminar (Laminar 2025) cluster inputs or extract signals but characterize input distributions, not resource attribution. AgentProf complements them by recursively assigning shared semantic responsibility over a preserved source-call tree and folding conserved resource measures into standard pprof.

**Cross-run agent analysis.** TraceProbe normalizes coding traces into canonical actions and process diagnostics; Graphectory builds cyclic action/navigation graphs and aggregates phase patterns (Shu et al. 2026; Liu et al. 2026a); Act·onomy supplies a fixed three-level behavior taxonomy; CHIEF builds task/subtask causal graphs for oracle-guided failure attribution (Gao et al. 2026; Wang et al. 2026c); Hodoscope compares behavior distributions to human review; and TraceGraph builds shared decision landscapes that guide recovery (Zhong, Saxena, and Raghunathan 2026; Nian et al. 2026). All six answer what an agent did and which step went wrong. AgentProf asks a profiler's question instead—how much of an additive resource a semantic category is responsible for—and that changes two things. First, responsibility is variable-depth and recovered from source content, where Act·onomy fixes three levels and TraceProbe canonicalizes a single action level. Second, one hierarchy is a projection surface: the same boundaries replay under count, token, and time widths with exact conservation, which cyclic graph and distribution aggregations do not provide. Native LLM and tool calls remain as drilldown leaves beneath every semantic frame, so the aggregate stays traceable to the evidence that produced it.

**Profiling and diagnosis.** pprof and flame graphs fold runtime stacks, whereas Pivot Tracing dynamically groups measurements across causally related events (Google 2024b; Gregg 2011; Mace, Roelke, and Fonseca 2015). AgentRx (Barke et al. 2026), TELBench (Wang et al. 2026a), AgentAtlas (Mazaheri and Mazaheri 2026), TrajAD (Liu et al. 2026b), and AgentFixer (Mulian et al. 2026) perform per-trajectory fault localization, CodeTracer reconstructs per-run states, and localization benchmarks score faulty steps, agents, or expert-supported perspectives (Li et al. 2026; Fan et al. 2026; Wang et al. 2026b; Chen et al. 2026; In et al. 2026). AgentProf applies the profiler's population-level principle to agents, annotating recurring semantic responsibility over completed trajectories without code identity and without treating runtime nesting as semantic truth.

## Conclusion

Agent observability needs profiling, not only debugging. AgentProf shows that the semantic operation stack model, driven by recursive operation segmentation, brings hierarchical attribution to agent trajectories. Against independent annotations, segmentation recovers human stage structure at 0.764 $B^3$ F1, profiles add localization signal on three complete public benchmarks, and one hierarchy replays across additive measures with exact conservation. A strong reader that re-reads a whole trajectory per question remains the best single-query localizer, confirming that profilers and debuggers are complementary: the profile answers population questions and guides the reader to the evidence worth opening. Multi-project evaluation and tracing-ecosystem integration are the most impactful next steps.

## References


Anthropic. 2025. Claude Code: An Agentic Coding Tool. https://code.claude.com/docs/en/overview.

Arize AI. 2024a. Arize Phoenix. https://arize.com/docs/phoenix.

Arize AI. 2024b. OpenInference Specification. https://arize-ai.github.io/openinference/spec/.

Bagga, A.; and Baldwin, B. 1998. Entity-Based Cross-Document Coreferencing Using the Vector Space Model. In *36th Annual Meeting of the Association for Computational*

*Linguistics and 17th International Conference on Computational Linguistics, Volume 1*, 79–85.

Balusu, K. C. 2026. AgentTelemetry: A Fault Detection Benchmark and Toolkit for LLM Agent Observability. In *Proceedings of the 3rd ACM International Conference on AI-Powered Software (AIware '26)*.

Barke, S.; Goyal, A.; Khare, A.; Singh, A.; Nath, S.; and Bansal, C. 2026. AgentRx: Diagnosing AI Agent Failures from Execution Trajectories. arXiv preprint arXiv:2602.02475.

Bouma, G. 2009. Normalized (Pointwise) Mutual Information in Collocation Extraction. In *From Form to Meaning: Processing Texts Automatically, Proceedings of the Biennial GSCL Conference 2009*, 31–40.

Bouzenia, I.; and Pradel, M. 2025. Understanding Software Engineering Agents: A Study of Thought-Action-Result Trajectories. In *2025 IEEE/ACM 40th International Conference on Automated Software Engineering (ASE)*, 2846–2857.

Chen, M.; Wang, J.; Mu, F.; Wang, Y.; Liu, Z.; Feng, H.; and Wang, Q. 2026. Seeing the Whole Elephant: A Benchmark for Failure Attribution in LLM-Based Multi-Agent Systems. arXiv preprint arXiv:2604.22708.

Datadog. 2025. LLM Observability. https://docs.datadoghq.com/llm_observability/.

Datadog. 2026. LLM Observability Patterns. https://docs.datadoghq.com/llm_observability/monitoring/patterns/.

Deng, X.; Gu, Y.; Zheng, B.; Chen, S.; Stevens, S.; Wang, B.; Sun, H.; and Su, Y. 2023. Mind2Web: Towards a Generalist Agent for the Web. In *Advances in Neural Information Processing Systems 36 (NeurIPS), Datasets and Benchmarks Track*.

Dong, L.; Lu, Q.; and Zhu, L. 2024. AgentOps: Enabling Observability of LLM Agents. arXiv preprint arXiv:2411.05285.

Fan, S.; Ye, X.; Huo, Y.; Chen, Z.-Y.; Guo, Y.; Yang, S.; Yang, W.; Ye, S.; Chen, J.; Chen, H.; Cong, X.; and Lin, Y. 2026. AgentProcessBench: Diagnosing Step-Level Process Quality in Tool-Using Agents. In *Proceedings of KDD*.

Gao, J.; Sun, K.; Huang, J.-t.; Van Koevering, K.; Ji, S.; Huang, H.; Shi, W.; Lu, Z.; Xiao, Z.; Khashabi, D.; and Dredze, M. 2026. How to Interpret Agent Behavior. arXiv:2605.13625.

Google. 2024a. Perfetto Trace Processor. https://perfetto.dev/docs/analysis/trace-processor.

Google. 2024b. pprof. https://github.com/google/pprof.

Gregg, B. 2011. Flame Graphs. https://www.brendangregg.com/flamegraphs.html.

In, Y.; Tanjim, M.; Subramanian, J.; Kim, S.; Bhattacharya, U.; Kim, W.; Park, S.; Sarkhel, S.; and Park, C. 2026. Rethinking Failure Attribution in Multi-Agent Systems: A Multi-Perspective Benchmark and Evaluation. arXiv:2603.25001.

Kendall, M. G. 1938. A New Measure of Rank Correlation. *Biometrika*, 30(1/2): 81–93.

Laminar. 2025. Signals: Structured Event Extraction from Traces. https://docs.lmnr.ai/signals/introduction.

LangChain. 2024. LangSmith Observability. https://docs.langchain.com/langsmith/observability.

LangChain. 2026. LangSmith Insights. https://docs.langchain.com/langsmith/insights.

Langfuse. 2024. Langfuse Observability. https://langfuse.com/docs/observability/overview.

Lewis, D. D.; Yang, Y.; Rose, T. G.; and Li, F. 2004. RCV1: A New Benchmark Collection for Text Categorization Research. *Journal of Machine Learning Research*, 5: 361–397.

Li, H.; Yao, Y.; Zhu, L.; Feng, R.; Ye, H.; Wang, J.; He, Y.; Zou, P.; Zhang, L.; Lei, X.; Huang, H.; Deng, K.; Sun, M.; Zhang, Z.; Ye, H.; and Liu, J. 2026. CodeTracer: Towards Traceable Agent States. arXiv:2604.11641.

Liu, S.; Chen, Y.; Krishna, R.; Sinha, S.; Ganhotra, J.; and Jabbarvand, R. 2026a. Process-Centric Analysis of Agentic Software Systems. *Proceedings of the ACM on Programming Languages*, 10(OOPSLA1): 1961–1988.

Liu, Y.; Zhang, C.; Han, Z.; Liu, H.; Wang, Y.; Yu, Y.; Wang, X.; and Yin, Y. 2026b. TrajAD: Trajectory Anomaly Detection for Trustworthy LLM Agents. arXiv preprint arXiv:2602.06443.

Lù, X. H.; Kazemnejad, A.; Meade, N.; Patel, A.; Shin, D.; Zambrano, A.; Stańczak, K.; Shaw, P.; Pal, C. J.; and Reddy, S. 2025. AgentRewardBench: Evaluating Automatic Evaluations of Web Agent Trajectories. arXiv preprint arXiv:2504.08942.

Ma, C.; Zhang, J.; Zhu, Z.; Yang, C.; Yang, Y.; Jin, Y.; Lan, Z.; Kong, L.; and He, J. 2024. AgentBoard: An Analytical Evaluation Board of Multi-turn LLM Agents. In *Advances in Neural Information Processing Systems*, volume 37, 74325–74362.

Mace, J.; Roelke, R.; and Fonseca, R. 2015. Pivot Tracing: Dynamic Causal Monitoring for Distributed Systems. In *Proceedings of the 25th Symposium on Operating Systems Principles*, 378–393.

MacQueen, J. B. 1967. Some Methods for Classification and Analysis of Multivariate Observations. In *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, volume 1, 281–297.

Mazaheri, P.; and Mazaheri, K. 2026. AgentAtlas: Beyond Outcome Leaderboards for LLM Agents. arXiv preprint arXiv:2605.20530.

McCallum, A.; and Nigam, K. 1998. A Comparison of Event Models for Naive Bayes Text Classification. In *AAAI-98 Workshop on Learning for Text Categorization*, 41–48.

Mulian, H.; Zeltyn, S.; Levy, I.; Galanti, L.; Yaeli, A.; and Shlomov, S. 2026. AgentFixer: From Failure Detection to Fix Recommendations in LLM Agentic Systems. In *Proceedings of the 1st International Workshop on Agentic Engineering (AGENT '26, co-located with ICSE)*.

Nian, J.; Chen, K.; Zhang, G.; Cao, Y.; and Jiang, Y. 2026. TraceGraph: Shared Decision Landscapes for Diagnosing and Improving Agent Trajectories. arXiv:2605.31308.

NVIDIA. 2026. Profiling and Performance Monitoring of NVIDIA NeMo Agent Toolkit Workflows. Official documentation.

OpenTelemetry. 2024. OpenTelemetry GenAI Semantic Conventions. https://github.com/open-telemetry/semantic-conventions-genai.

OpenTelemetry. 2026. OpenTelemetry Profiles. Official specification.

Qwen Team. 2026. Qwen3.6-27B: Flagship-Level Coding in a 27B Dense Model. Official model card.

Robertson, S. 2008. A New Interpretation of Average Precision. In *Proceedings of the 31st Annual International ACM SIGIR conference on Research and Development in Information Retrieval*, 689–690.

Rosenberg, A.; and Hirschberg, J. 2007. V-Measure: A Conditional Entropy-Based External Cluster Evaluation Measure. In *Proceedings of the 2007 Joint Conference on Empirical Methods in Natural Language Processing and Computational Natural Language Learning*, 410–420.

Ruokolainen, T.; Kohonen, O.; Sirts, K.; Grönroos, S.-A.; Kurimo, M.; and Virpioja, S. 2016. A Comparative Study of Minimally Supervised Morphological Segmentation. *Computational Linguistics*, 42(1): 91–120.

Sajadi, A.; Nguyen, T.; Huynh, K.; Parra, E.; and Chatterjee, P. 2026. TraceView: Interactive Visualization of Agentic Program Repair Trajectories. arXiv:2606.22110.

Shu, R.; Chong, C. Y.; Zhou, X.; Peng, Y.; Wu, Z.; Han, X.; Zhuang, Z.; Yuan, G.; and Wang, Y. 2026. What Resolve Rate Hides: Trajectory Structure Diagnostics for Coding Agents. arXiv:2607.06184.

Spearman, C. 1904. The Proof and Measurement of Association between Two Things. *The American Journal of Psychology*, 15(1): 72–101.

Wang, J.; Feng, Z.; Wu, J.; Li, R.; Xie, Q.; Ren, Y.; Zhu, H.; Han, X.; Meng, F.; Feng, J.; and Liu, J. 2026a. Where Do Deep-Research Agents Go Wrong? Span-Level Error Localization in Agent Trajectories. arXiv preprint arXiv:2606.02060.

Wang, J.; Hou, J.; Wang, F.; Jian, P.; Bao, C.; and Lv, Z. 2026b. HINTBench: Horizon-Agent Intrinsic Non-Attack Trajectory Benchmark. arXiv preprint arXiv:2604.13954.

Wang, R.; Jansen, P.; Côté, M.-A.; and Ammanabrolu, P. 2022. ScienceWorld: Is Your Agent Smarter than a 5th Grader? In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, 11279–11298.

Wang, Y.; Wu, W.; Wang, J.; and Wang, Q. 2026c. From Flat Logs to Causal Graphs: Hierarchical Failure Attribution for LLM-based Multi-Agent Systems. arXiv:2602.23701.

Wang, Y.; Zhang, J.; Cai, T.; Liu, Z.; Sun, Q.; Sun, Z.; Wu, Z.; Dong, M.; Zheng, M.; Yin, X.; and Zhu, Y. 2026d. From Agent Traces to Trust: A Survey of Evidence Tracing and Execution Provenance in LLM Agents. arXiv preprint arXiv:2606.04990.

WukLab. 2025. OSWorld-Human. https://github.com/WukLab/osworld-human.

Xie, T.; Zhang, D.; Chen, J.; Li, X.; Zhao, S.; Cao, R.; Hua, T. J.; Cheng, Z.; Shi, D.; Tong, J.; Lu, K.-W.; Garg, V.; Wang, Y.; Dai, Z.; Li, F.; Bisk, Y.; and Yu, T. 2024. OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. In *Advances in Neural Information Processing Systems 37 (NeurIPS), Datasets and Benchmarks Track*.

Yang, J.; Jimenez, C. E.; Wettig, A.; Lieret, K.; Yao, S.; Narasimhan, K.; and Press, O. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems 37 (NeurIPS)*.

Zheng, L.; Chiang, W.-L.; Sheng, Y.; Zhuang, S.; Wu, Z.; Zhuang, Y.; Lin, Z.; Li, Z.; Li, D.; Xing, E. P.; Zhang, H.; Gonzalez, J. E.; and Stoica, I. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems 36 (NeurIPS), Datasets and Benchmarks Track*.

Zheng, Y.; Hu, Y.; Yu, T.; and Quinn, A. 2025. AgentSight: System-Level Observability for AI Agents Using eBPF. In *Proceedings of the 4th Workshop on Practical Adoption Challenges of ML for Systems (PACMI '25, co-located with SOSP)*.

Zhong, Z.; Saxena, S.; and Raghunathan, A. 2026. Hodoscope: Unsupervised Monitoring for AI Misbehaviors. arXiv:2604.11072.